Course Code : MCS-201
Course Title : Programming in C and PYTHON
Assignment Number : PGDCA(I)/201/Assignment/2024
Maximum Marks : 100
Weightage : 30% **www.ignousite.com**
Last Date of Submission : 30th April, 2024 (for January session)
31st October, 2024 (for July session)

SECTION-A (C-Programming)

Q1: Briefly discuss the concept of "Call by value" and "Call by reference". Give example code in C for each. Support your code with suitable comments.

Ans. "Call by value" and "Call by reference" are two different ways of passing arguments to functions in programming languages. These concepts refer to how the values of arguments are passed to functions and how changes to those values within the function affect the original variables.

Call by Value: In Call by Value, the actual value of the argument is passed to the function. The function receives a copy of the argument, and any modifications made to the parameter inside the function do not affect the original value outside the function.

Example:

```
#include <stdio.h>

// Function that performs Call by Value
void incrementByValue(int x) {
   x = x + 1;
   printf("Inside function: x = %d\n", x);
}

int main() {
   int num = 5;

   printf("Before function call: num = %d\n", num);

   // Call the function with num as argument
   incrementByValue(num);

   printf("After function call: num = %d\n", num);

   return 0;
}
```

Output:

```
Before function call: num = 5
Inside function: x = 6
After function call: num = 5
```

Call by Reference: In Call by Reference, the memory address (reference) of the actual argument is passed to the function. Any changes made to the parameter inside the function directly affect the original value outside the function.

Example:

```
#include <stdio.h>

// Function that performs Call by Reference
void incrementByReference(int *x) {
   *x = *x + 1;
   printf("Inside function: *x = %d\n", *x);
}

int main() {
   int num = 5;

   printf("Before function call: num = %d\n", num);

   // Call the function with the address of num as argument
   incrementByReference(&num);

   printf("After function call: num = %d\n", num);

   return 0;
}
```

Output:

```
Before function call: num = 5
Inside function: *x = 6
After function call: num = 6
```

Q2: Briefly discuss the relation between Pointers and Arrays, giving suitable example. Write a program in C, to print transpose of a 2D matrix entered by a user. Support your program with suitable comments.

Ans. Relation between Pointers and Arrays: In C, arrays and pointers are closely related. An array name can be used as a pointer to the first element of the array. When you pass an array to a function, you are actually passing a pointer to the first element of the array. This is why array subscript notation is equivalent to pointer arithmetic.

Example:

```
#include <stdio.h>


int main() {
   int arr[] = {1, 2, 3, 4, 5};
   int *ptr;

   // Pointers and Arrays Relationship

   // Assigning the address of the first element of the array to the pointer
   ptr = arr;

   // Accessing array elements using array notation
   printf("Array element at index 2: %d\n", arr[2]);

   // Accessing array elements using pointer notation
   printf("Array element at index 2 using pointer: %d\n", *(ptr + 2));

   // Using a loop to access array elements through the pointer
   printf("Array elements using a loop and pointer:\n");
   for (int i = 0; i < 5; i++) {
      printf("%d ", *(ptr + i));
   }

   return 0;
}
```

Output:

```
Array element at index 2: 3
Array element at index 2 using pointer: 3
Array elements using a loop and pointer:
1 2 3 4 5
```

Print transpose of a 2D matrix: C program that takes a 2D matrix as input from the user and prints its transpose.

```
#include <stdio.h>

#define MAX_ROWS 10

#define MAX_COLS 10

int main() {

   int matrix[MAX_ROWS][MAX_COLS];

   int transpose[MAX_COLS][MAX_ROWS];

   int rows, cols;

   // Input: Number of rows and columns

   printf("Enter the number of rows (1 to 10): ");

   scanf("%d", &rows);


   printf("Enter the number of columns (1 to 10): ");

   scanf("%d", &cols);


   // Input: Matrix elements

   printf("Enter the elements of the matrix:\n");

   for (int i = 0; i < rows; i++) {

      for (int j = 0; j < cols; j++) {

         printf("Element [%d][%d]: ", i + 1, j + 1);

         scanf("%d", &matrix[i][j]);

      }
```

```
    }

    // Calculating transpose
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    // Output: Original Matrix
    printf("\nOriginal Matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }

    // Output: Transpose Matrix
    printf("\nTranspose Matrix:\n");
    for (int i = 0; i < cols; i++) {
        for (int j = 0; j < rows; j++) {
            printf("%d\t", transpose[i][j]);
        }
        printf("\n");
    }
  return 0;
}
```

Output:

```
Enter the number of rows (1 to 10): 4
Enter the number of columns (1 to 10): 4
Enter the elements of the matrix:
Element [1][1]: 2
Element [1][2]: 2
Element [1][3]: 3
Element [1][4]: 3
Element [2][1]: 4
Element [2][2]: 4
Element [2][3]: 5
Element [2][4]: 5
Element [3][1]: 1
Element [3][2]: 1
Element [3][3]: 2
Element [3][4]: 2
Element [4][1]: 3
Element [4][2]: 3
Element [4][3]: 4
Element [4][4]: 4
Original Matrix:
2       2       3       3
4       4       5       5
1       1       2       2
3       3       4       4

Transpose Matrix:
2       4       1       3
2       4       1       3
3       5       2       4
3       5       2       4
```

Q3: Write an algorithm to find the slope of a line segment whose end point coordinates are (x1, y1) and (x2, y2). The algorithm gives output whether the slope is positive, negative or zero. Transform your algorithm to C program.

Note : Slope of line segment = (y2 – y1) / (x2 – x1)

Ans. Algorithm to Find the Slope of a Line Segment:

1. Input the coordinates (x1, y1) and (x2, y2) of the two endpoints.
2. Calculate the difference in y-coordinates: delta_y = y2 - y1.
3. Calculate the difference in x-coordinates: delta_x = x2 - x1.
4. If delta_x is zero, the line is vertical, and the slope is undefined.
   - Output "Slope is undefined."
5. If delta_x is not zero:
   a. Calculate the slope: slope = delta_y / delta_x.
   b. If slope is positive, output "Slope is positive."

c. If slope is negative, output "Slope is negative."
d. If slope is zero, output "Slope is zero."

C Program:

```
#include <stdio.h>

int main() {
   // Input: Endpoint coordinates
   float x1, y1, x2, y2;

   printf("Enter the coordinates of the first endpoint (x1, y1): ");
   scanf("%f %f", &x1, &y1);

   printf("Enter the coordinates of the second endpoint (x2, y2): ");
   scanf("%f %f", &x2, &y2);

   // Calculate differences in coordinates
   float delta_y = y2 - y1;
   float delta_x = x2 - x1;

   // Check if the line is vertical
   if (delta_x == 0) {
      printf("Slope is undefined (vertical line).\n");
   } else {
      // Calculate the slope
      float slope = delta_y / delta_x;

      // Output the slope type
      if (slope > 0) {
         printf("Slope is positive.\n");
      } else if (slope < 0) {
         printf("Slope is negative.\n");
      } else {
         printf("Slope is zero.\n");
      }
   }

   return 0;
}
```

Output:

```
Enter the coordinates of the first endpoint (x1, y1): 5 10
Enter the coordinates of the second endpoint (x2, y2): 8 12
Slope is positive.

Enter the coordinates of the first endpoint (x1, y1): 5 4
```

```
Enter the coordinates of the second endpoint (x2, y2): 4 5
Slope is negative.

Enter the coordinates of the first endpoint (x1, y1): 5 3
Enter the coordinates of the second endpoint (x2, y2): 7 3
Slope is zero.
```

Q4: Write an algorithm to find the HCF (Highest Common Factor) of the two numbers entered by a user. Transform your algorithm to a C program, support your program with suitable comments.

Ans. Algorithm to Find the Highest Common Factor (HCF) of Two Numbers:

1. Input the two numbers num1 and num2 from the user.
2. Set divisor to the smaller of the two numbers.
3. Repeat the following steps until divisor is greater than 1:
   a. If both num1 and num2 are divisible by divisor without any remainder, then divisor is the HCF.
   b. If found, exit the loop and output the HCF.
   c. Otherwise, decrement divisor by 1 and repeat the process.
4. If no common factor is found other than 1, then the HCF is 1.

C Program:

```
#include <stdio.h>

// Function to find the HCF of two numbers
int findHCF(int num1, int num2) {
   int divisor = (num1 < num2) ? num1 : num2;

   while (divisor > 1) {
      if (num1 % divisor == 0 && num2 % divisor == 0) {
         return divisor;  // Found the HCF
      }
      divisor--;
   }

   return 1;  // If no common factor is found other than 1
}

int main() {
   int num1, num2;

   // Input: Two numbers from the user
   printf("Enter the first number: ");
   scanf("%d", &num1);

   printf("Enter the second number: ");
   scanf("%d", &num2);
```

```
    // Call the function to find HCF and output the result
    int hcf = findHCF(num1, num2);
    printf("The HCF of %d and %d is %d.\n", num1, num2, hcf);

    return 0;
}
```

Output:

```
Enter the first number: 5
Enter the second number: 8
The HCF of 5 and 8 is 1.
```

SECTION-B (PYTHON-Programming)

Q1: Discuss the connect( ) method of MySQL. Connector interface. List the arguments involved with connect( ) method. Write Python code to create database student_DB and to connect to student_DB (make suitable assumptions wherever necessary).

Ans. Connect() method: The connect() method is part of the MySQL Connector/Python library, which is a MySQL driver for Python that provides an interface to interact with MySQL databases. The connect() method is used to establish a connection to a MySQL server. This method is part of the mysql.connector module, and it returns a connection object that can be used to interact with the database.

List the arguments of connect() method:

The connect() method accepts several arguments, some of the common ones include:

1. **host**: The host where the MySQL server is running.
2. **user:** The MySQL user to authenticate as.
3. **password**: The password for the MySQL user.
4. **database**: The name of the database to connect to.
5. **port:** The TCP/IP port number to use for the connection.
6. **charset:** The character set to use for the connection.

Python Code to Create Database and Connect: Assuming you have the MySQL Connector/Python library installed, here's an example code to create a database named student_DB and connect to it. Please note that you need to have the necessary permissions to create a database:

```
import mysql.connector


# Assuming MySQL server is running locally

host = "localhost"

user = "your_username"  # Replace with your MySQL username

password = "your_password"  # Replace with your MySQL password
```

```
# Connect to MySQL server (no database specified)
connection = mysql.connector.connect(
    host=host,
    user=user,
    password=password
)

# Create a cursor object to execute SQL queries
cursor = connection.cursor()

# Create a new database named 'student_DB'
database_name = "student_DB"
create_db_query = f"CREATE DATABASE IF NOT EXISTS {database_name}"
cursor.execute(create_db_query)

print(f"Database '{database_name}' created successfully.")

# Connect to the 'student_DB' database
connection = mysql.connector.connect(
    host=host,
    user=user,
    password=password,
    database=database_name
)

print(f"Connected to database '{database_name}'.")

# Close the cursor and connection
```

```
cursor.close()

connection.close()
```

Q2: What are Pandas ? Write steps to import, read and print a CSV file using Pandas. Also, transform your steps in to suitable code in Python.

Ans. Introduction to Pandas: Pandas is a powerful open-source data manipulation and analysis library for Python. It provides data structures such as Series and DataFrame, along with a collection of functions to manipulate and analyze structured data seamlessly. Pandas is widely used in data science, machine learning, and data analysis tasks due to its simplicity and flexibility.

Steps to Import, Read, and Print a CSV file using Pandas:

1. **Import Pandas:** The first step is to import the Pandas library in your Python script or Jupyter Notebook. This is done using the import statement:

   ```
   import pandas as pd
   ```

Here, pd is a common alias for Pandas, making it easier to reference throughout your code.

2. **Read CSV File**: Use the read_csv() function from Pandas to read the data from a CSV (Comma Separated Values) file. This function automatically converts the data into a DataFrame, a two-dimensional tabular data structure.

   ```
   df = pd.read_csv('your_file.csv')
   ```

3. **Print the DataFrame**: To view the contents of the DataFrame, use the print() function or simply type the variable name in the Python environment.

   ```
   print(df)
   ```

Alternatively, you can use Jupyter Notebook to display the DataFrame directly by typing the variable name in a cell.

Python Code: Here is a complete example code illustrating the above steps:

```
import pandas as pd

# Step 1: Import Pandas

# Step 2: Read CSV File

df = pd.read_csv('your_file.csv')  # Replace with your CSV file path

# Step 3: Print the DataFrame

print(df)
```

Make sure to replace 'your_file.csv' with the actual path or URL of your CSV file.

Example with Explanation:

Let's assume you have a CSV file named example_data.csv with the following contents:

Name, Age, Gender

Sunil, 25, Female

Sonam, 30, Male

Sandeep, 22, Male

Here's how the Python code looks:

```
import pandas as pd


# Import Pandas

# Step 1: Import Pandas


# Read CSV File

# Step 2: Read CSV File

df = pd.read_csv('example_data.csv')  # Replace with your CSV file path


# Print the DataFrame

# Step 3: Print the DataFrame

print(df)
```

When you run this script, it will read the CSV file into a DataFrame and print the contents:

```
     Name     Age    Gender
0    Sunil      25     Female
1    Sonam   30    Male
2   Sandeep   22    Male
```

The DataFrame is a tabular representation of your CSV data, where each row corresponds to a record, and each column corresponds to a field. This allows for easy manipulation, analysis, and visualization of your data using Pandas functionalities.

Q3: Write steps to create a package. Apply these steps to create a package named volume and create 3 modules in it named cube, cuboid and sphere, having function to calculate volume of the cube, cuboid and sphere respectively. Import the modules defined in the package and use the defined functions for calculation of volume respectively.

Ans. Creating a package involves organizing your Python code into a directory structure with a special __init__.py file, which marks the directory as a package. Below are the steps to create a package named "volume" and three modules inside it named "cube.py," "cuboid.py," and "sphere.py."

Steps to Create a Package:

1. **Create a directory for your package:**

   mkdir volume

2. **Inside the "volume" directory, create an __init__.py file:**

   touch volume/__init__.py

This file can be empty but is necessary to treat the directory as a Python package.

3. **Create module files inside the "volume" directory:**

Create cube.py:

```
def cube_volume(side_length):
    return side_length ** 3
```

Create cuboid.py:

```
def cuboid_volume(length, width, height):
    return length * width * height
```

Create sphere.py:

```
import math
def sphere_volume(radius):
    return (4/3) * math.pi * radius ** 3
```

4. **Using the Package in a Separate Script:**

Create a Python script in the same directory as your "volume" directory. For example, create a file named calculate_volume.py:

```
# calculate_volume.py

from volume import cube, cuboid, sphere

# Example usage of the functions
```

```
cube_volume_result = cube.cube_volume(3)

cuboid_volume_result = cuboid.cuboid_volume(2, 4, 5)

sphere_volume_result = sphere.sphere_volume(3)


# Print the results

print(f"Volume of the cube: {cube_volume_result}")

print(f"Volume of the cuboid: {cuboid_volume_result}")

print(f"Volume of the sphere: {sphere_volume_result}")
```

We import the functions from the cube, cuboid, and sphere modules inside the "volume" package. Then, we use these functions to calculate the volume of a cube, cuboid, and sphere, respectively.

Run the Script:

python calculate_volume.py

This will execute the script, and you should see output similar to the following:

```
Volume of the cube: 27

Volume of the cuboid: 40

Volume of the sphere: 113.09733552923254
```

These steps illustrate how to organize your code into a package structure and use the functions defined in the package modules in a separate script. Packages in Python are a way to structure and organize your code, making it modular and easy to maintain.

Q4: What does *map( )* function do ? Write a program in Python to print the cube of the numbers present in the list, by using *map( )* function.

Ans. map() function: The map() function in Python applies a specified function to all the items in an input iterable (such as a list) and returns an iterator that produces the results. The syntax of the map() function is:

```
map(function, iterable, ...)
```

- **function:** The function to apply to each item in the iterable.
- **iterable:** The iterable (e.g., a list) whose elements will be processed by the function.

Program to Print the Cube of Numbers using map() function:

```
# Function to calculate the cube of a number
def cube(x):
   return x ** 3

# List of numbers
numbers = [1, 2, 3, 4, 5]
```

```
# Using map() to apply the cube function to each element in the list
cubes = map(cube, numbers)

# Converting the result to a list (as map() returns an iterator)
cubes_list = list(cubes)

# Printing the original numbers and their cubes
print("Original Numbers:", numbers)
print("Cubes of Numbers:", cubes_list)
```

Output:

```
Original Numbers: [1, 2, 3, 4, 5]
Cubes of Numbers: [1, 8, 27, 64, 125]
```

In this program:

1. We define a cube function that takes a number as input and returns its cube.
2. We create a list of numbers (numbers).
3. We use the map() function to apply the cube function to each element in the numbers list.
4. The result is an iterator, so we convert it to a list (cubes_list) to print the final output.

This program demonstrates how the map() function simplifies the process of applying a function to each element of a list, in this case, calculating the cube of each number. The result is a new list containing the cubes of the original numbers.